

Le Mécanisme d'Attention

Du goulot d'étranglement au Transformer

Plan

- ① pourquoi les RNN ne suffisent pas
- ② Le mécanisme d'attention (Bahdanau, 2014)
- ③ Généraliser l'attention : Requêtes, Clés et Valeurs
- ④ La formule mathématique de l'attention
- ⑤ La self-attention : quand la séquence s'observe elle-même
- ⑥ Le Transformer : assembler les pièces

Pourquoi les RNN ne suffisent pas

Le problème général des RNN

Quelle que soit l'architecture (seq2seq, seq2vec, vec2seq, encodeur-décodeur), les RNN souffrent du même problème fondamental :

État caché = mémoire de taille fixe

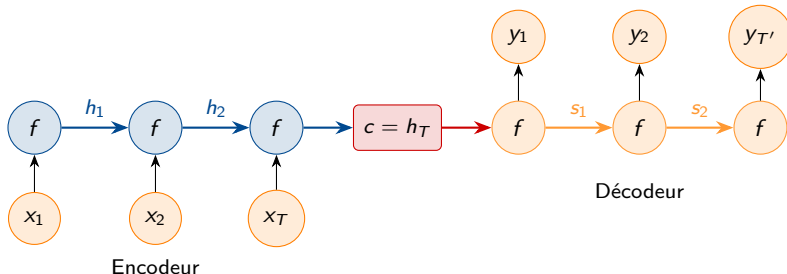
L'état caché h_t doit résumer **tout le passé** en un vecteur de taille fixe. Plus la séquence est longue, plus l'information ancienne est « écrasée ».

Le cas le plus critique est l'architecture **Encodeur-Décodeur**, où un unique vecteur $c = h_T$ doit capturer toute l'entrée.

Architecture Encodeur-Décodeur

Le modèle **Seq2Seq** repose sur deux réseaux récurrents :

- 1 **Encodeur** : lit la séquence d'entrée $X = (x_1, \dots, x_T)$ et produit des états cachés $h_1, \dots, h_T$.
- 2 **Vecteur de contexte** : le dernier état $c = h_T$ résume *toute* l'entrée.
- 3 **Décodeur** : génère la sortie $Y = (y_1, \dots, y_{T'})$ à partir de c .



Problème de la représentation de taille fixe

Le goulot d'étranglement

Toute l'information d'une séquence de longueur variable est compressée dans un **unique vecteur de taille fixe** $c \in \mathbb{R}^d$.

Cas A : « Bonjour » (1 mot)

→ 1 mot dans un vecteur de 512 dim.

OK : peu d'information.

Cas B : Paragraphe de 50 mots

→ 50 mots dans le **même** vecteur de 512 dim.

Problème : compression avec perte.

Le vecteur de contexte agit comme un **filtre passe-bas** : il retient l'idée générale mais perd les détails spécifiques, surtout ceux du **début** de la séquence.

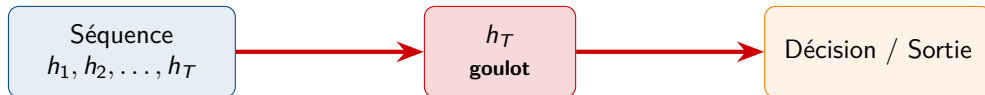
Gradients évanescents et oubli catastrophique

Lors de la **rétropropagation à travers le temps** (BPTT), le gradient pour une entrée au pas $t = 1$ doit remonter depuis le pas T :

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

- Si la plus grande valeur propre de W_h est < 1 : **évanouissement exponentiel**.
Exemple : $0,9^{50} \approx 0,005$ — le gradient est quasi nul.
- Le modèle « oublie » le début de la phrase en arrivant à la fin.
- Les LSTM atténuent le problème (portes, cellule c_t), mais la **capacité de la mémoire reste fixe**.

Résumé du problème



Toute la séquence compressée dans un seul vecteur !

Ce problème touche **toutes** les architectures RNN :

- **Seq2Vec** : h_T résume toute la phrase pour la classification.
- **Encodeur-Décodeur** : h_T résume toute l'entrée pour le décodeur.

Question

Comment accéder directement à **chaque** état caché $h_1, \dots, h_T$, plutôt qu'au seul dernier vecteur h_T ?

Le mécanisme d'attention

Bahdanau, 2014

Bahdanau, Cho et Bengio (2014) proposent une modification radicale :

Hypothèse

Au lieu de ne garder que h_T , le décodeur devrait avoir accès à **tous** les états cachés $h_1, h_2, \dots, h_T$.

- L'encodeur devient une **bibliothèque** où chaque h_i représente le i -ème mot et son contexte local.
- Le décodeur dispose d'un **accès aléatoire** (*random access*) à toute l'entrée.

Le mécanisme d'alignement

Donner 50 états cachés crée un nouveau problème : **surcharge d'information**.

Le décodeur doit savoir **quels mots sont pertinents** à chaque pas.

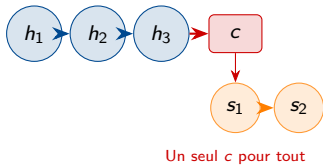
- 1 **Comparer** : l'état précédent du décodeur s_{t-1} avec chaque h_j de l'encodeur.
- 2 **Scorer** : attribuer un score d'alignement e_{tj} à chaque h_j .
- 3 **Normaliser** : Softmax $\rightarrow$ poids d'attention α_{tj} qui somment à 1.
- 4 **Agréger** : somme pondérée des états cachés.

$$c_t = \sum_{j=1}^T \alpha_{tj} h_j$$

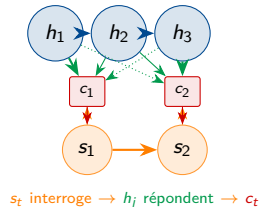
Le vecteur de contexte c_t est **dynamique** : il change à chaque mot généré !

Visualisation : contexte statique vs. dynamique

Seq2Seq classique



Avec attention



Attention de Bahdanau

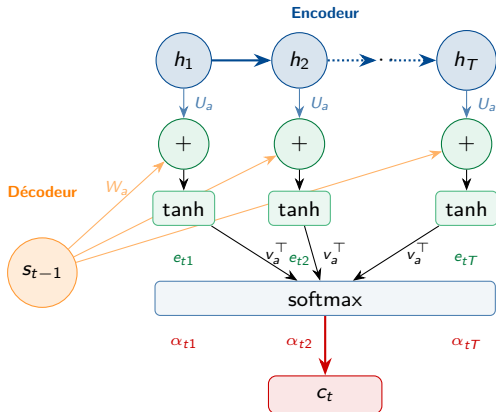
Score (réseau feedforward) :

$$e_{tj} = v_a^\top \tanh(W_a s_{t-1} + U_a h_j)$$

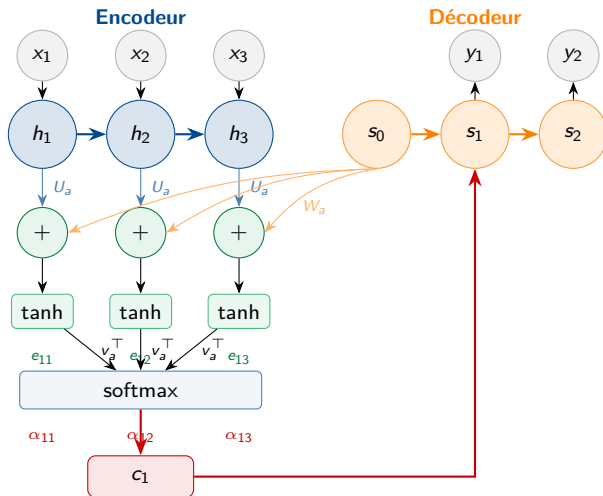
- W_a, U_a : projections apprises.
- v_a : vecteur $\rightarrow$ scalaire.
- **Additive** : somme avant tanh.

$$\alpha_{tj} = \text{softmax}(e_{tj})$$

$$c_t = \sum_{j=1}^T \alpha_{tj} h_j$$



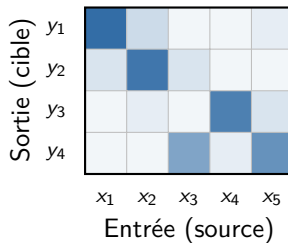
Flux complet



Flux : $s_0 \xrightarrow{W_a} + h_j \xrightarrow{U_a} \tanh \xrightarrow{v_a^T} \text{softmax} \rightarrow c_1 \rightarrow \text{Décodeur} \rightarrow s_1$

Cartes d'attention (heatmaps)

Les poids d'attention α_{tj} peuvent être visualisés comme une **matrice** :



- Fournit de l'**interprétabilité** : on voit *où* le modèle regarde.

Requêtes, Clés et Valeurs

Généraliser l'attention

Abstraction de l'attention : Q, K, V

En 2017, **Vaswani et al.** (“Attention Is All You Need”) abstraient l'attention en un mécanisme général de **recherche d'information** :



Imaginez une **bibliothèque** :

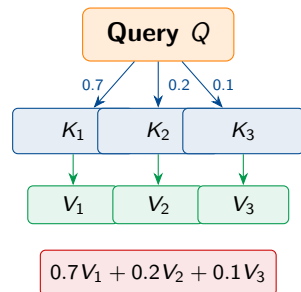
- **Key** (K) : la fiche du catalogue
(titre, catégorie, mots-clés)
- **Value** (V) : le contenu du livre
(le texte intégral)
- **Query** (Q) : votre question
« Comment fonctionne l'IA ? »

Base de données classique (recherche dure) :

→ renvoie le livre dont $K = Q$ exactement.

Attention (recherche douce) :

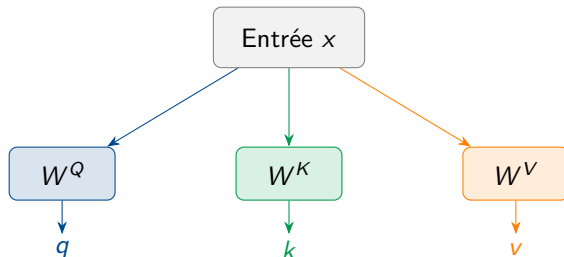
→ renvoie une **combinaison pondérée** de livres selon la similarité entre Q et chaque K .



Dérivation à partir du vecteur d'entrée

Q , K et V sont dérivés du **même vecteur d'entrée** x (représentation numérique d'un mot) par des **projections linéaires apprises** :

$$q = x \cdot W^Q, \quad k = x \cdot W^K, \quad v = x \cdot W^V$$



Pourquoi séparer ?

L'information pour **appairier** (chercher) est différente de celle pour **comprendre** (contenu). Les matrices W^Q , W^K , W^V séparent l'**espace d'adressage** de l'**espace de contenu**.

La formule mathématique

Scaled Dot-Product Attention

La formule

Maintenant que nous disposons de Q , K et V , comment calculer l'attention ?

Vaswani et al. (*Attention Is All You Need*, 2017) proposent la formule suivante :

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

Décomposons-la en **cinq étapes** :

- 1 **Projection** : $X \cdot W^Q$, $X \cdot W^K$, $X \cdot W^V$
- 2 **Similarité** : $QK^{\top}$
- 3 **Mise à l'échelle** : $/\sqrt{d_k}$
- 4 **Normalisation** : $\text{softmax}(\cdot)$
- 5 **Agrégation** : $\cdot V$

Étape 1 : le produit scalaire QK^T

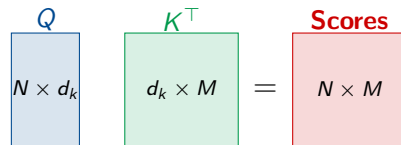
Entrées :

- $Q \in \mathbb{R}^{N \times d_k}$ (queries)
- $K \in \mathbb{R}^{M \times d_k}$ (keys)

Résultat : $QK^T \in \mathbb{R}^{N \times M}$

Intuition : le produit scalaire mesure l'**alignement géométrique**.

- $q \cdot k > 0$: même direction $\rightarrow$ pertinent.
- $q \cdot k = 0$: orthogonal $\rightarrow$ non lié.
- $q \cdot k < 0$: opposé.



Étape 2 : le facteur d'échelle $\sqrt{d_k}$

Pourquoi diviser par $\sqrt{d_k}$?

Plus la dimension d_k est grande, plus les produits scalaires $q \cdot k$ prennent des valeurs **extrêmes**. Le Softmax sature alors et les **gradients s'évanouissent**.

Scores modérés

$$\begin{aligned}\text{softmax}([2, 1, -1]) \\ \approx [0,7, 0,2, 0,1]\end{aligned}$$

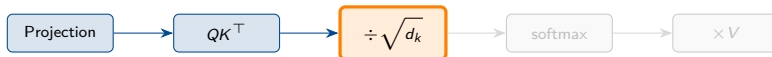
Distribution douce $\rightarrow$ gradients corrects

Scores extrêmes

$$\begin{aligned}\text{softmax}([20, 10, -10]) \\ \approx [1,0, 0,0, 0,0]\end{aligned}$$

Saturation $\rightarrow$ gradients ≈ 0

Diviser par $\sqrt{d_k}$ ramène les scores dans une plage raisonnable, quel que soit d_k .



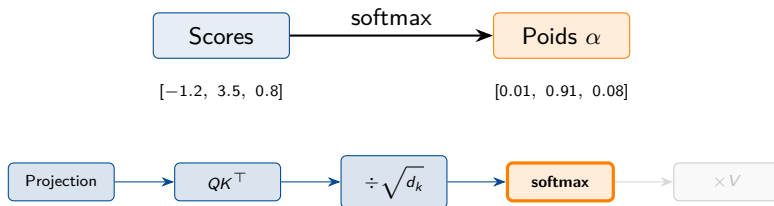
Étape 3 : le Softmax

Le Softmax convertit les scores bruts en une **distribution de probabilité** :

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- **Propriété 1** : toutes les sorties sont positives.
- **Propriété 2** : la somme vaut 1.

Résultat : les **poids d'attention** α_{ij} , qui déterminent « combien » de chaque Value récupérer.



Étape 4 : la somme pondérée ... V

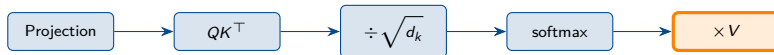
On multiplie les poids d'attention par la matrice de Values :

$$\underbrace{\text{Sortie} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)}_{N \times M} \cdot \underbrace{V}_{M \times d_v} = \underbrace{\text{Sortie}}_{N \times d_v}$$

Pour chaque query, on obtient une **moyenne pondérée** des Values :

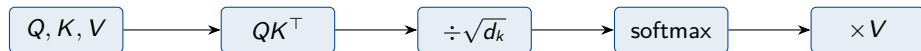
- Les mots avec un **score élevé** (signal) sont amplifiés.
- Les mots avec un **score faible** (bruit) sont filtrés.

$$\begin{array}{c} \text{Poids} \\ N \times M \end{array} \times \begin{array}{c} V \\ M \times d_v \end{array} = \begin{array}{c} \text{Sortie} \\ N \times d_v \end{array}$$



Récapitulatif des étapes

Étape	Opération	Intuition	Impact
1	$X \cdot W^Q, W^K, W^V$	Créer adresses et contenu	Sépare les rôles
2	$QK^\top$	Mesurer la similarité	Trouve le contexte
3	$\div \sqrt{d_k}$	Stabiliser la variance	Évite grad. évanescents
4	$\text{softmax}(\cdot)$	Convertir en probabilités	Poids qui somment à 1
5	$\cdot V$	Récupérer le contenu	Représentation contextualisée



Exemple numérique pas à pas

Soit $d_k = 4$, une seule query q et trois keys k_1, k_2, k_3 :

$$q = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad k_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad k_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}, \quad k_3 = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

Étape 1 – Produits scalaires :

$$q \cdot k_1 = 1, \quad q \cdot k_2 = 1, \quad q \cdot k_3 = 2$$

Étape 2 – Mise à l'échelle ($\sqrt{4} = 2$) :

$$\frac{1}{2} = 0,5, \quad \frac{1}{2} = 0,5, \quad \frac{2}{2} = 1,0$$

Étape 3 – Softmax :

$$\alpha \approx (0,23, 0,23, 0,54)$$

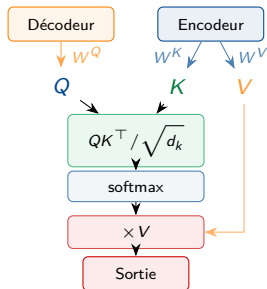
Étape 4 : Sortie = $0,23 v_1 + 0,23 v_2 + 0,54 v_3$ (forte contribution de v_3).

La self-attention

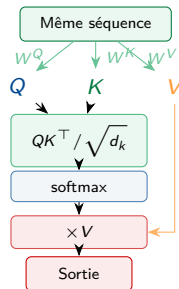
Quand la séquence s'observe elle-même

Cross-attention vs. Self-attention

Cross-attention



Self-attention



La seule différence : **d'où viennent** Q , K , V . Le calcul est **identique**.

Pourquoi s'auto-observer ?

Le mot « banque » a plusieurs sens. La self-attention le **désambiguïse** :

Phrase A

« J'ai déposé de l'**argent** à la **banque**. »

→ Query de « banque » s'aligne avec
« argent », « déposé ».

→ Représentation enrichie : **institution
financière**.

Phrase B

« La **banque** de **données** contient tous les
fichiers. »

→ Query de « banque » s'aligne avec
« données », « fichiers ».

→ Représentation enrichie : **base de
stockage**.

On produit des **représentations contextuelles dynamiques**.

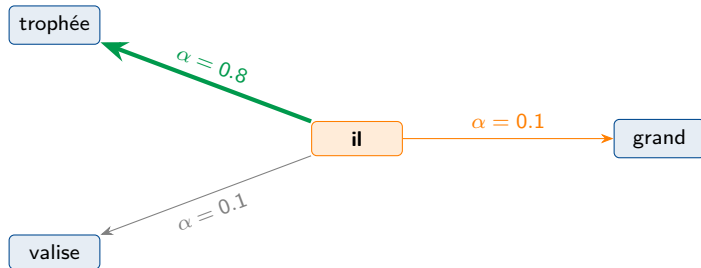
Résolution des références

« Le trophée ne rentrait pas dans la valise parce qu'**il** était trop **grand**. »

- À quoi fait référence « il » ?
- La self-attention compare « il » avec « trophée » et « valise ».
- « grand » et « ne rentrait pas » créent un contexte : → **le trophée**.

Si la phrase finissait par « trop **petite** » :

→ les poids se déplaceraient vers **la valise**.



Le Transformer

Assembler les pièces

« Attention Is All You Need » (Vaswani et al., 2017)

Construisons l'encodeur (1/5) : Entrée

On commence par l'**entrée** de l'encodeur :

- Chaque mot est converti en un **vecteur numérique**.
- On y ajoute un **encodage positionnel** pour représenter l'ordre des mots.

C'est le point de départ : la séquence sous forme de vecteurs.



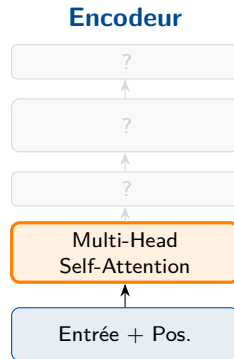
Construisons l'encodeur (2/5) : Multi-Head Self-Attention

La première couche est une **Multi-Head Self-Attention**.

Chaque mot regarde **tous les autres** mots de la séquence pour enrichir sa représentation.

Nouveau composant

Qu'est-ce que le **Multi-Head** Attention ?



Plutôt qu'un seul ensemble Q, K, V , on utilise h ensembles (**têtes**) en parallèle :

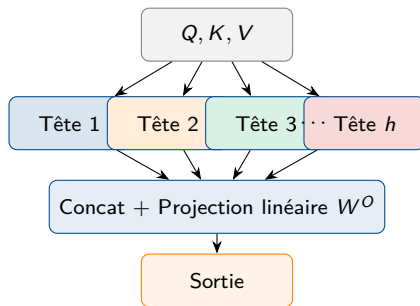
- **Tête 1** : relations grammaticales (sujet–verbe).
- **Tête 2** : définitions sémantiques.
- **Tête 3** : résolution de pronoms (« il » → « robot »).
- ...

Chaque tête capture un **type de relation différent** dans la séquence.

Multi-Head Attention

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$



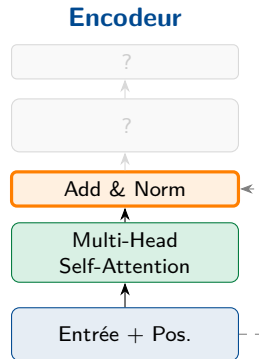
Construisons l'encodeur (3/5) : Add & Norm

Après chaque sous-couche, on applique **Add & Norm** :

- **Add** : connexion résiduelle ($x + \text{Sublayer}(x)$).
- **Norm** : Layer Normalization.

Nouveau composant

Qu'est-ce que Add & Norm ?



Connexions résiduelles et Layer Norm

Connexion résiduelle

$$\text{output} = x + \text{Sublayer}(x)$$

- Permet aux gradients de **circuler directement**.
- Facilite l'entraînement de modèles profonds.

Layer Normalization

Normalise les activations **au sein de chaque mot** (pas sur le batch) :

$$\text{LN}(x) = \gamma \cdot \frac{x - \mu}{\sigma + \epsilon} + \beta$$

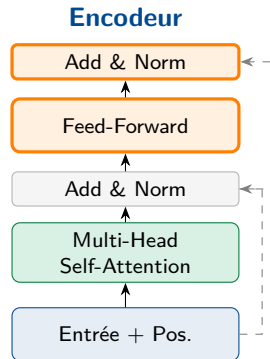
- Stabilise l'entraînement.

Construisons l'encodeur (4/5) : Feed-Forward

Après l'attention, un **réseau Feed-Forward** est appliqué à chaque position indépendamment :

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

- Deux couches linéaires avec ReLU.
- Suivi d'un second **Add & Norm**.

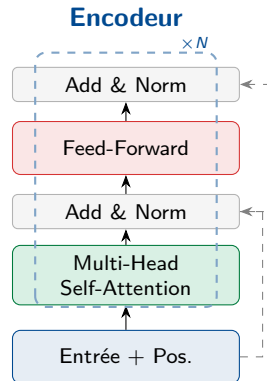


Construisons l'encodeur (5/5)

L'encodeur est composé de N **blocs identiques** empilés :

- 1 Multi-Head Self-Attention
- 2 Add & Norm
- 3 Feed-Forward
- 4 Add & Norm

La sortie du dernier bloc est la **représentation contextuelle** de la séquence.



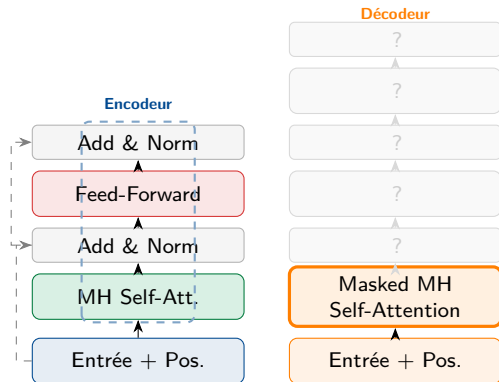
Construisons le décodeur (1/3) : Masked Self-Attention

Le décodeur commence comme l'encodeur, mais avec une différence clé : l'attention est **masquée**.

Lors de la génération, le mot t ne doit pas voir les mots **futurs** ($t + 1, t + 2, \dots$).

Nouveau composant

Pourquoi et comment **masquer** ?



Le masquage causal (Masked Attention)

Dans le décodeur, lors de la génération, on ne doit pas voir les mots **futurs** (sinon le modèle **triche** en lisant la réponse au lieu de la prédire).

On applique un masque aux scores avant
le softmax :

$$\text{score}_{ij} = \begin{cases} q_i \cdot k_j & \text{si } j \leq i \\ -\infty & \text{si } j > i \end{cases}$$

$-\infty$ dans le softmax $\rightarrow$ poids = 0.

	k_1	k_2	k_3	k_4
q_1	e_{11}	$-\infty$	$-\infty$	$-\infty$
q_2	e_{21}	e_{22}	$-\infty$	$-\infty$
q_3				
q_4				

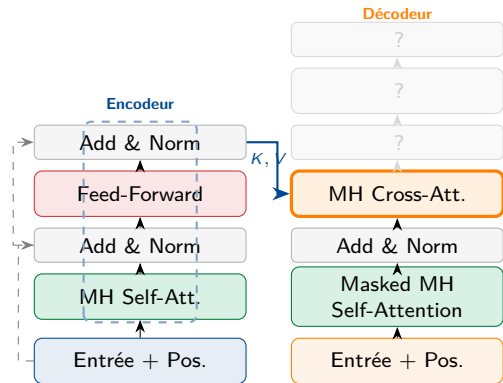
visible masqué

Construisons le décodeur (2/3) : Cross-Attention

Après la self-attention masquée (+ Add & Norm), le décodeur effectue une **cross-attention** avec l'encodeur :

- Q vient du **décodeur**.
- K, V viennent de l'**encodeur**.

C'est ce mécanisme qui permet au décodeur de **consulter la séquence source**.

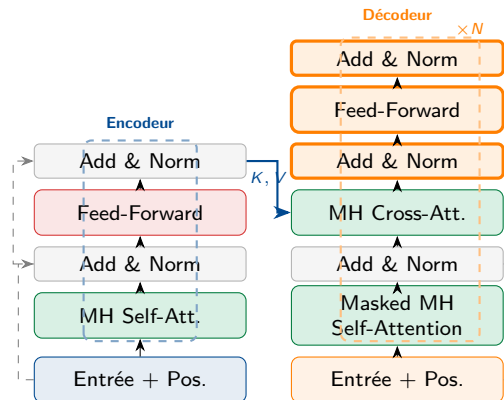


Construisons le décodeur (3/3) : Feed-Forward

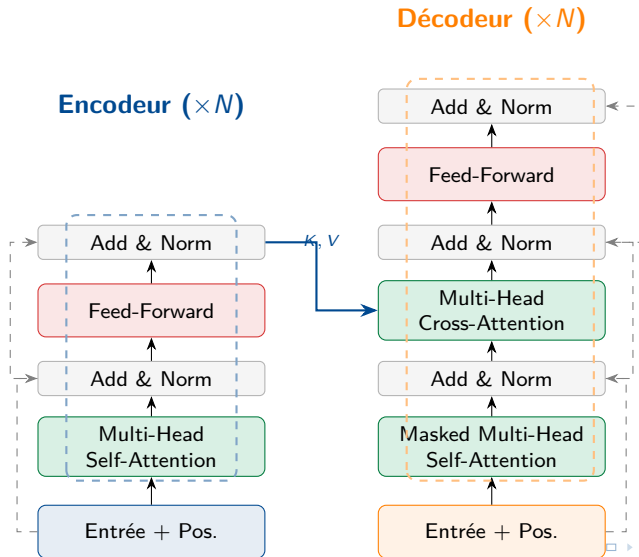
Comme dans l'encodeur, le décodeur termine par :

- **Add & Norm** après la cross-attention.
- **Feed-Forward** (même architecture que l'encodeur).
- **Add & Norm** final.

Le tout est répété N fois, comme pour l'encodeur.

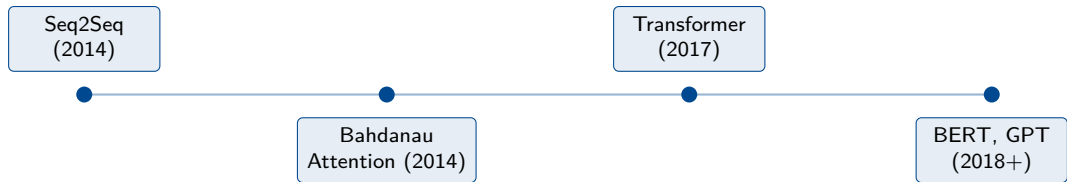


Architecture complète du Transformer



- **Goulot d'étranglement** : les Seq2Seq compressent toute une séquence en un vecteur fixe, perdant l'information sur les phrases longues.
- **Attention (Bahdanau)** : le décodeur a accès à **tous** les états de l'encodeur via une somme pondérée dynamique.
- **Q, K, V** : abstraction générale de l'attention comme un système de recherche douce.
- **Scaled Dot-Product Attention** : la division par $\sqrt{d_k}$ stabilise les gradients.
- **Self-Attention** : élimine la récurrence. Chaque mot regarde directement tous les autres.
- **Multi-Head Attention** : capture des relations de nature différente en parallèle.
- **Transformer** : l'architecture qui a permis BERT, GPT et l'ère de l'IA générative.

Chronologie



Questions ?